

Lecture 4: Free-Space Management

- The file system must know which storage blocks are available for future allocation.
- Free-space management affects allocation speed, fragmentation and metadata overhead.
- Main techniques: bitmaps, linked lists, grouping and counting.

Learning Objectives

- Explain why free-space management is required.
- Construct and interpret a free-space bitmap.
- Explain linked-list, grouping and counting methods.
- Analyze allocation time and metadata overhead.

Why Track Free Space?

- A new file or an expanding file needs one or more free blocks.
- Deleted/truncated files return blocks to the free-space pool.
- The allocator must avoid allocating the same block to multiple files.
- It may also need to find **contiguous** blocks or suitable extents.

- One bit represents the allocation state of one block.
- Example convention: 1 = allocated, 0 = free.

Block	0	1	2	3	4	5	6	7	8	9
State	1	0	0	1	1	0	1	0	0	1

- Free blocks: 1, 2, 5, 7, 8.

Bitmap Space Overhead

- If there are N blocks and one bit is used per block:

$$\text{bitmap size} = \frac{N}{8} \text{ bytes}$$

- Example: 1,000,000 blocks require:

$$1,000,000/8 = 125,000 \text{ bytes} \approx 122.1 \text{ KiB}$$

- Bitmaps are convenient for finding runs of consecutive free blocks.

Linked Free-Space List

Free head $\rightarrow 5 \rightarrow 9 \rightarrow 14 \rightarrow 21 \rightarrow \text{NULL}$

- Each free block can point to another free block.
- Allocation removes blocks from the list.
- Deallocation inserts blocks into the list.
- Finding a long contiguous run may be expensive.

- A free block stores addresses of multiple free blocks.
- The last address points to another group.

Group 1: [7, 9, 12, 15] → Group 2: [21, 22, 30, 31] → ...

This reduces the number of pointer traversals compared with a simple linked list.

- Consecutive free blocks can be represented by a starting block and a count.

Start	Count
100	8
250	14
400	3

- (100, 8) means blocks 100–107 are free.
- Particularly efficient when free space occurs in long runs.

Allocation and Deallocation

- ➊ Request arrives: e.g., 6 blocks.
- ➋ Free-space structure is searched.
- ➌ Suitable blocks/extents are selected.
- ➍ Metadata is updated.
- ➎ On deletion, the released blocks are returned to the free-space structure.

Important: allocation policy can influence future fragmentation.

Comparison

Technique	Advantage	Limitation
Bitmap	Easy status representation; good for runs	Bitmap storage and scanning
Linked list	Simple	Slow search for suitable runs
Grouping	Faster group discovery	More complex
Counting	Compact for contiguous runs	Less effective with highly scattered free blocks

- **Galvin, 10th Ed.:** Chapter 14 – Free-Space Management: bit vector, linked list, grouping, counting and space maps.
- **Stallings, 9th Ed.:** Chapter 18 – Secondary Storage Management and Unix/Linux file management.

- Free-space management is essential for safe and efficient block allocation.
- Bitmap methods provide compact state representation.
- Counting is attractive when free space occurs in runs.
- The best technique depends on allocation patterns and workload.

Exercise: Given a bitmap of 32 blocks, identify all free runs of length ≥ 3 and determine whether a 5-block request can be satisfied contiguously.